

Lecture 21 and 22: See the Network: CLI Tools, Wireshark, and pyshark Touch the Network: Scapy, Mininet, and Sockets

This report explores the transition from theoretical networking concepts to professional practice as detailed in Lectures 21 and 22 of EC 441. While previous units focused on the internal mechanics of protocols, these sessions emphasize observing, interacting with, and building real network traffic

Lecture 21 introduces the essential toolkit for diagnosing and monitoring live network states. The core philosophy is that the protocols studied all semester Ethernet, IP, TCP, UDP, and DNS are actively running on modern machines and can be inspected using standard Linux tools
Command-Line Diagnostic Tools

Network engineers utilize specific CLI tools that provide a direct lens into the protocol stack:

- ping: Primarily used for reachability and Round-Trip Time (RTT) estimation. It connects back to L17 (ICMP Echo Request/Reply) and L19 (RTT estimation), where the variance in results provides the signal for TCP's retransmission timers.
- traceroute: Reveals the sequence of routers to a destination by exploiting TTL decrement and ICMP Time Exceeded messages. Significant RTT jumps often indicate crossing Autonomous System (AS) boundaries or intercontinental links.
- ip: Replaces older utilities like ifconfig to manage interfaces, the IP routing table, and the ARP cache (ip neigh).
- ss: The "socket statistics" tool is critical for observing live TCP state. By using the -tipm flags, students can watch the congestion window (cwnd) and ssthresh evolve in real-time during data transfers, turning abstract L20 concepts into observable data.
- dig: A DNS resolution tool that explores the DNS hierarchy, highlighting cache lifetimes (TTL) and the difference between recursive and authoritative resolvers.

2. Packet Analysis: Wireshark and Pyshark

While tcpdump allows for raw command-line packet capture, deep inspection requires more sophisticated tools.

- Wireshark: Provides a graphical interface to follow TCP streams and inspect protocol layers (Ethernet → IP → UDP → DNS). It is ideal for diagnosing individual conversations but limited for bulk analysis.
- Pyshark: This Python wrapper treats a packet capture as a dataset. It allows for programmatic analysis, such as extracting all DNS query names or plotting TCP RTT over time to see the raw signal that TCP's EWMA filters into srtt

Lecture 22 shifts the focus from observation to active participation through packet crafting, topology emulation, and application development.

1. Custom Packet Crafting with Scapy

Scapy is described as "Wireshark in reverse". Instead of parsing bytes into objects, it allows users to build Python objects and write them as bytes onto the wire.

- Layer Stacking: Scapy uses the / operator to encapsulate protocols (e.g., IP() / ICMP()).
- Raw Sockets: Because Scapy bypasses the kernel's normal socket layer to send custom-built frames, it requires root privileges.
- Traceroute Implementation: A simple 10-line Python loop in Scapy can replicate traceroute by manually incrementing the TTL field in IP headers.

2. Topology Emulation with Mininet

Mininet uses Linux network namespaces to create a real network in miniature on a single kernel.

- Why Mininet?: Standard loopback interfaces lack the delay and loss found in real networks. Mininet allows students to create bottleneck topologies with controllable bandwidth and loss (e.g., 1%), which triggers TCP mechanisms like fast retransmit and multiplicative decrease. This makes the AIMD (Additive Increase, Multiplicative Decrease) arc visible in tools like ss.

3. The Socket API

The socket API is the fundamental interface for all networked applications. The lecture highlights the critical asymmetries between TCP (SOCK_STREAM) and UDP (SOCK_DGRAM):

- TCP: Requires a connection handshake (listen, accept, connect) and provides a guaranteed byte stream.
- UDP: Is stateless and message-preserving, requiring no setup but offering no retransmission or ordering guarantees.
- Key "Traps": Application developers must use SO_REUSEADDR to avoid "address already in use" errors during the TIME_WAIT state. Additionally, recv(N) should be treated as a hint rather than a guarantee of receiving exactly N bytes.

Conclusion: The "payoff" of these lectures is the ability to close the read and write loop. By writing a custom application with sockets, emulating a network path with Mininet, and capturing the resulting traffic with tcpdump or pyshark, students can observe the entire protocol stack in action. A single captured .pcap file now represents the synthesis of the entire semester: Ethernet frames (L08) carrying IP datagrams (L14–17) and TCP segments (L18–20), all supporting application bytes generated by the student's own code.